Práctica 1

Elementos básicos de la programación

Alumno: Montoya García José Antonio
Docente: Jose Carlos Gallegos Mariscal
Grupo: 941

Universidad Autónoma de Baja California
Facultad de Ingeniería, Arquitectura y Diseño

Introducción

En este proyecto vamos a analizar cómo funciona una **cola de impresión** dentro de un sistema informático.

En muchas oficinas o centros de trabajo, varias personas envían documentos a imprimir al mismo tiempo. Sin embargo, una impresora solo puede imprimir **un documento a la vez**, por lo que necesita organizar los trabajos que recibe.

Para resolver este problema se utiliza una estructura de datos llamada **cola**, que sigue el principio:

FIFO (First In, First Out)
El primer trabajo que llega es el primero en procesarse.

En esta práctica se desarrolló un sistema en **lenguaje C** que simula una cola de impresión.
El sistema fue implementado en **tres versiones diferentes** con el objetivo de analizar el manejo de memoria y el comportamiento de las estructuras de datos.

Las versiones desarrolladas fueron:

- **Sesión 1:** Implementación de una cola estática.
- **Sesión 2:** Implementación de una cola dinámica usando memoria dinámica.
- **Sesión 3:** Simulación completa del proceso de impresión.

El objetivo principal es comprender la diferencia entre utilizar **memoria estática** y **memoria dinámica**, así como aplicar estructuras de datos para resolver problemas reales.

Demostración de Conceptos

En esta sección se describen los conceptos utilizados en el desarrollo de los programas que implementan el sistema de cola de impresión.

Sesión 1 — Cola Estática

En la primera implementación se desarrolló una **cola estática**, lo que significa que el tamaño máximo de la cola está definido desde el inicio del programa.

Este tipo de implementación utiliza **arreglos** para almacenar los trabajos de impresión.

La ventaja de este enfoque es que es **simple de implementar**, pero tiene la desventaja de que el número de elementos está limitado.

Definición de librerías y estructuras

Aquí se muestra la parte del código donde se definen las librerías y estructuras necesarias para el funcionamiento del programa.

pp_01.cpp > __unnamed_struct_223e_3 > prioridad

```cpp
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
////////////////////////////////////////////////////////////////////////////
#define MAX_USER 32
#define MAX_DOC 48
#define MAX_JOBS 10
////////////////////////////////////////////////////////////////////////////
typedef enum { NORMAL = 0, URGENTE = 1 } Prioridad_t;
typedef enum { EN_COLA = 0, IMPRIMIENDO = 1, COMPLETADO = 2, CANCELADO = 3 } Estado_t;
////////////////////////////////////////////////////////////////////////////
typedef struct
{
    int id;
    char usuario[MAX_USER];
    char documento[MAX_DOC];
    int paginas_total;
    int paginas_restantes;
    int copias;
    Prioridad_t prioridad;
    Estado_t estado;
    int ms_por_pagina;
} PrintJob_t;
////////////////////////////////////////////////////////////////////////////
typedef struct
{
    PrintJob_t data[MAX_JOBS];
    int size;
} QueueStatic_t;
////////////////////////////////////////////////////////////////////////////
```

Función para limpiar salto de línea

Esta función elimina el salto de línea que se genera cuando se utiliza `fgets`, permitiendo limpiar correctamente las cadenas de texto ingresadas por el usuario.

```c
////////////////////////////////////////////////////////////////////////
void trim_newline(char* str)
{
    str[strcspn(str, "\r\n")] = 0;
}
////////////////////////////////////////////////////////////////////////
int main()
```

Estructura del trabajo de impresión

El programa define una estructura llamada **PrintJob_t**, la cual almacena la información de cada trabajo de impresión.

```c
////////////////////////////////////////////////////////////////////////
void qs_init(QueueStatic_t* q)
{
    q->size = 0;
}
////////////////////////////////////////////////////////////////////////
int qs_is_empty(const QueueStatic_t* q)
{
    return q->size == 0;
}
////////////////////////////////////////////////////////////////////////
int qs_is_full(const QueueStatic_t* q)
{
    return q->size == MAX_JOBS;
}
////////////////////////////////////////////////////////////////////////
int qs_enqueue(QueueStatic_t* q, PrintJob_t job)
{
    if (qs_is_full(q))
    {
        return 0;
    }
    q->data[q->size] = job;
    q->size++;
    return 1;
}

int qs_peek(const QueueStatic_t* q, PrintJob_t* out)
{
    if (qs_is_empty(q))
    {
        printf("La cola esta vacia.\n");
        return 0;
    }
    *out = q->data[0];
```

Esta estructura incluye:

- Identificador del trabajo
- Nombre del usuario
- Nombre del documento
- Número total de páginas
- Páginas restantes
- Número de copias
- Prioridad del trabajo
- Estado del trabajo
- Tiempo de impresión por página

Estructura de la cola estática

La cola de impresión se administra mediante una estructura que contiene un arreglo de trabajos y un contador de elementos.

Estructura cola estática

Operaciones implementadas

El sistema permite realizar las siguientes operaciones:

- Inicializar la cola
- Verificar si la cola está vacía
- Verificar si la cola está llena
- Agregar un trabajo (enqueue)
- Visualizar el siguiente trabajo (peek)
- Procesar un trabajo (dequeue)
- Mostrar todos los trabajos

Ejecución del código — Sesión 1

Opción 1

```
PROBLEMS    TERMINAL    ···                pp_01  +

--- MENU COLA DE IMPRESION (ESTATICA) ---
1. Agregar trabajo (enqueue)
2. Ver siguiente trabajo (peek)
3. Procesar trabajo (dequeue)
4. Listar cola
5. Salir
Seleccione una opcion: 1
Usuario: tony
Documento: practicas
Total de paginas: 10
Trabajo agregado con ID: 1
```

Opción 2

```
PROBLEMS    TERMINAL    ···                pp_01  +

--- MENU COLA DE IMPRESION (ESTATICA) ---
1. Agregar trabajo (enqueue)
2. Ver siguiente trabajo (peek)
3. Procesar trabajo (dequeue)
4. Listar cola
5. Salir
Seleccione una opcion: 2
Siguiente: [ID 1] tony - practicas (10 pags)
```

Opción 3

```
--- MENU COLA DE IMPRESION (ESTATICA) ---
1. Agregar trabajo (enqueue)
2. Ver siguiente trabajo (peek)
3. Procesar trabajo (dequeue)
4. Listar cola
5. Salir
Seleccione una opcion: 3
Procesando y removiendo: [ID 1] practicas
```

Opción 4

```
--- MENU COLA DE IMPRESION (ESTATICA) ---
1. Agregar trabajo (enqueue)
2. Ver siguiente trabajo (peek)
3. Procesar trabajo (dequeue)
4. Listar cola
5. Salir
Seleccione una opcion: 4
Cola vacia []
```

Sesión 2 — Cola Dinámica

En la segunda implementación se utilizó **memoria dinámica** para almacenar los trabajos de impresión.

Esto permite que la cola pueda **crecer dinámicamente** según el número de trabajos enviados por los usuarios.

Para lograr esto se utilizan funciones como:

Código de implementación dinámica

```c
typedef struct
{
    int id;
    char usuario[MAX_USER];
    char documento[MAX_DOC];
    int paginas_total;
    int paginas_restantes;
    int copias;
    Prioridad_t prioridad;
    Estado_t estado;
    int ms_por_pagina;
} PrintJob_t;

////////////////////////////////////////////////////////
typedef struct Node
{
    PrintJob_t job;
    struct Node* next;
} Node_t;
////////////////////////////////////////////////////////
typedef struct
{
    Node_t* head; // Frente de la cola
    Node_t* tail; // Final de la cola
    int size;
} QueueDynamic_t;
```

Estructura de nodo

Cada trabajo se almacena dentro de un nodo de una lista enlazada.

```c
    PrintJob_t job;
    struct Node* next;
} Node_t;
////////////////////////////////////////////////////////
typedef struct
{
    Node_t* head; // Frente de la cola
    Node_t* tail; // Final de la cola
    int size;
} QueueDynamic_t;
```

Cada nodo contiene:

- Un trabajo de impresión
- Un puntero al siguiente nodo

Esto permite crear una **lista enlazada**.

Función enqueue

La función enqueue se encarga de agregar un nuevo trabajo al final de la cola.

```c
int qd_enqueue(QueueDynamic_t* q, PrintJob_t job)
{
    Node_t* nuevo = (Node_t*)malloc(sizeof(Node_t));
    if (nuevo == NULL)
    {
        printf("Error: No hay memoria suficiente (Heap Full).\n");
        return 0;
    }
    nuevo->job = job;
    nuevo->next = NULL;

    if (qd_is_empty(q))
    {
        q->head = q->tail = nuevo;
    }
    else
    {
        q->tail->next = nuevo;
        q->tail = nuevo;
    }
    q->size++;
    return 1;
}
////////////////////////////////////////////////////////////////////////
int qd_dequeue(QueueDynamic_t* q, PrintJob_t* out)
{
    if (qd_is_empty(q))
    {
        printf("Cola vacia.\n");
        return 0;
    }
    Node_t* temp = q->head;
    *out = temp->job;

    q->head = q->head->next;
```

Función dequeue

La función dequeue elimina el primer trabajo de la cola y libera la memoria utilizada.

```
}
////////////////////////////////////////////////////////////////////////
int qd_dequeue(QueueDynamic_t* q, PrintJob_t* out)
{
    if (qd_is_empty(q))
    {
        printf("Cola vacia.\n");
        return 0;
    }
    Node_t* temp = q->head;
    *out = temp->job;

    q->head = q->head->next;
```

Liberación de memoria

Antes de que el programa termine, se libera toda la memoria utilizada para evitar **fugas de memoria**.

```
////////////////////////////////////////////////////////////////////////
int qd_peek(const QueueDynamic_t* q, PrintJob_t* out)
{
    if (qd_is_empty(q))
    {
        printf("Cola vacia.\n");
        return 0;
    }
    *out = q->head->job;
    return 1;
}
////////////////////////////////////////////////////////////////////////
```

Ejecución del código — Sesión 2

Opción 1

```
PROBLEMS    TERMINAL    ···                pp_02

--- MENU COLA DINAMICA (SESION 2) ---
1. Agregar trabajo (malloc)
2. Ver siguiente (peek)
3. Procesar trabajo (free)
4. Listar cola din├ímica
5. Salir y limpiar memoria
Seleccione una opcion: 1
Usuario: Pepe aguilar
Documento: Peso pluma
Total de paginas: 32
Agregado exitosamente a la memoria Heap.
```

Opción 2

```
--- MENU COLA DINAMICA (SESION 2) ---
1. Agregar trabajo (malloc)
2. Ver siguiente (peek)
3. Procesar trabajo (free)
4. Listar cola din├ímica
5. Salir y limpiar memoria
Seleccione una opcion: 2
Siguiente: [1] Ejercicios
```

Opción 3

```
PROBLEMS    TERMINAL    ···                pp_

--- MENU COLA DINAMICA (SESION 2) ---
1. Agregar trabajo (malloc)
2. Ver siguiente (peek)
3. Procesar trabajo (free)
4. Listar cola din├ímica
5. Salir y limpiar memoria
Seleccione una opcion: 3
Procesado y liberado: Ejercicios
```

Opción 4

```
--- MENU COLA DINAMICA (SESION 2) ---
1. Agregar trabajo (malloc)
2. Ver siguiente (peek)
3. Procesar trabajo (free)
4. Listar cola dinámica
5. Salir y limpiar memoria
Seleccione una opcion: 4
--- LISTA DE TRABAJOS (DINAMICA) ---
ID: 2 | Peso pluma  | Pags: 32
```

Sesión 3 — Simulación de Cola de Impresión

En la tercera implementación se desarrolló una **simulación más completa del sistema de impresión**.

El sistema permite:

- Registrar trabajos de impresión
- Almacenarlos en una cola
- Simular el proceso de impresión página por página
- Mostrar estadísticas del sistema

Código del sistema de simulación

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h> // Para sleep()
////////////////////////////////////////////////////////////////////////
#define MAX_USER 32
#define MAX_DOC 48
////////////////////////////////////////////////////////////////////////
typedef enum { NORMAL = 0, URGENTE = 1 } Prioridad_t;
typedef enum { EN_COLA = 0, IMPRIMIENDO = 1, COMPLETADO = 2, CANCELADO = 3 } Estado_t;
////////////////////////////////////////////////////////////////////////
typedef struct
{
    int id;
    char usuario[MAX_USER];
    char documento[MAX_DOC];
    int paginas_total;
    int paginas_restantes;
    Prioridad_t prioridad;
    Estado_t estado;
} PrintJob_t;
////////////////////////////////////////////////////////////////////////
typedef struct Node
{
    PrintJob_t job;
    struct Node* next;
} Node_t;
////////////////////////////////////////////////////////////////////////
typedef struct
{
    Node_t *head, *tail;
    int count;
    // Variables para Mejora: Estadísticas
    int total_procesados;
    int total_paginas_impresas;
} PrintQueue_t;
```

Estructura de la cola de impresión

```c
void simular_impresion(PrintQueue_t* q)
{
    if (q->head == NULL)
    {
        printf("Nada que imprimir.\n");
        return;
    }

    PrintJob_t t;
    while (q->head != NULL)
    {
        q_dequeue(q, &t);
        t.estado = IMPRIMIENDO; // Cambio de estado

        printf("\n>>> [ID %d] Iniciando: %s (Usuario: %s)\n", t.id, t.documento, t.usuario);

        for (int p = 1; p <= t.paginas_total; p++)
        {
            printf("\r   Imprimiendo pagina %d/%d... ", p, t.paginas_total);


            printf("[");
            for(int i=0; i<10; i++)
            {
                if(i < (p*10/t.paginas_total)) printf("#"); else printf(".");
            }
            printf("]");

            fflush(stdout);
            usleep(300000);
        }

        t.estado = COMPLETADO;
        q->total_procesados++;
        q->total_paginas_impresas += t.paginas_total;
        printf("\n✓ Trabajo finalizado.\n");
```

Simulación del proceso de impresión

Durante la simulación el sistema muestra el avance de impresión página por página.

```c
int main()
{
    PrintQueue_t q;
    q_init(&q);
    int opcion = 0;
    int proximo_id = 1;
    char buffer[128];

    do
    {
        printf("\n=== SISTEMA DE IMPRESION PROFESIONAL (SESION 3) ===\n");
        printf("1. Nuevo trabajo\n2. Ver cola\n3. INICIAR SIMULACION\n4. Ver Estadisticas (Mejora)\n5.
        printf("Seleccion: ");
        if (fgets(buffer, sizeof(buffer), stdin)) opcion = (int)strtol(buffer, NULL, 10);

        switch (opcion)
        {
            case 1:
            {
                PrintJob_t nj;
                nj.id = proximo_id++;
                printf("Usuario: "); fgets(nj.usuario, MAX_USER, stdin); trim_newline(nj.usuario);
                printf("Archivo: "); fgets(nj.documento, MAX_DOC, stdin); trim_newline(nj.documento);
                printf("Paginas: "); fgets(buffer, sizeof(buffer), stdin); nj.paginas_total = (int)strtol(bu

                if(nj.paginas_total > 0)
                {
                    nj.paginas_restantes = nj.paginas_total;
                    nj.estado = EN_COLA;
                    nj.prioridad = NORMAL;
                    q_enqueue(&q, nj);
                }
                else
```

Ejecución del código — Sesión 3

Opción 1

```
PROBLEMS    OUTPUT    TERMINAL    ...    pp_03

=== SISTEMA DE IMPRESION PROFESIONAL (SESION 3) ===
1. Nuevo trabajo
2. Ver cola
3. INICIAR SIMULACION
4. Ver Estadisticas (Mejora)
5. Salir
Seleccion: 1
Usuario: Peso pluma
Archivo: jaze
Paginas: 25
```

Opción 2

```
PROBLEMS    OUTPUT    TERMINAL    ···    pp_03

=== SISTEMA DE IMPRESION PROFESIONAL (SESION 3) ===
1. Nuevo trabajo
2. Ver cola
3. INICIAR SIMULACION
4. Ver Estadisticas (Mejora)
5. Salir
Seleccion: 2
[1] jaze | Pags: 25 | Estado: 0
```

Opción 3

```
=== SISTEMA DE IMPRESION PROFESIONAL (SESION 3) ===
1. Nuevo trabajo
2. Ver cola
3. INICIAR SIMULACION
4. Ver Estadisticas (Mejora)
5. Salir
Seleccion: 3

>>> [ID 1] Iniciando: jaze (Usuario: Peso pluma )
   Imprimiendo pagina 25/25... [##########]
Γ£ö Trabajo finalizado.

--- COLA VACIA: Todos los trabajos han sido procesados ---
```

Opción 4

```
=== SISTEMA DE IMPRESION PROFESIONAL (SESION 3) ===
1. Nuevo trabajo
2. Ver cola
3. INICIAR SIMULACION
4. Ver Estadisticas (Mejora)
5. Salir
Seleccion: 4

--- ESTADISTICAS DE LA SESION ---
Trabajos completados: 1
Total paginas impresas: 25
```

Conclusión

A través del desarrollo de estos programas fue posible comprender diversos conceptos importantes de programación en lenguaje **C** y de **estructuras de datos**.

En el primer programa se implementó una **cola estática**, lo que permitió entender cómo almacenar datos utilizando arreglos con tamaño fijo.

En el segundo programa se implementó una **cola dinámica** utilizando listas enlazadas y memoria dinámica mediante `malloc` y `free`.

Finalmente, en el tercer programa se desarrolló una **simulación completa del proceso de impresión**, lo que permitió observar cómo los sistemas reales administran los trabajos enviados a una impresora.

En conjunto, esta práctica permitió reforzar conocimientos sobre:

- Estructuras
- Enumeraciones
- Punteros
- Manejo de memoria dinámica
- Listas enlazadas
- Estructuras de datos tipo cola

Además, permitió comprender cómo estos conceptos pueden aplicarse para resolver problemas reales dentro de sistemas informáticos.